오픈소스 기반 데이터분석

03강

데이터 분석을 위한 파이썬 프로그래밍 2

컴퓨터과학과 정재화교수



학습목차

1. 파이썬 문법 요소
2. 함수형 프로그래밍



01

파이썬 문법 요소



언패킹의 개념

언패킹(unpacking)의 정의

- 시퀀스 자료형(리스트, 튜플, 문자열 등)에서 여러 개의 값을 개별 변수에 분리하여 할당하는 기능
- 하나의 데이터 묶음에서 개별 요소들을 추출하여 각각의 변수에 직접 할당

문법형식1

$$변수_1, \ldots, 변수_{n-1}, 변수_n = 시퀀스_객체$$

문법형식2

$$변수_1, \ldots, 변수_{n-1}^*, 변수_n = 시퀀스_객체$$

언패킹 활용

시퀀스 언패킹

```
1 rgb = [255, 128, 0]
2 red, green, blue = rgb
3 print(f"빨강: {red}, 초록: {green}, 파랑: {blue}")
4 word = "ABC"
5 first, second, third = word
6 print(f"첫번째 문자: {first}, 두번째 문자: {second},
          세번째 문자: {third}")
```

확장 언패킹

```
1 monthly_sales = [1200, 1350, 1420, 1500, 1300, 1580,
                   1620, 1700, 1800, 1850, 1900, 2000]
2 first, *middle, last = monthly_sales
3 print(f"첫 달(1월) 판매액: {first}")
4 print(f"마지막 달(12월) 판매액: {last}")
5 print(f"중간 달(2~11월)의 판매액: {middle}")
```

언더스코어

- 시퀀스 데이터는 일반적으로 반복문을 사용하여 처리
- 언더스코어(_)는 반복 변수의 값이 필요하지 않을 때 이를 대체하는 역할

```
1 for _ in range(5):
2     print("Hello, Data Analysis with Open Source!")
```

```
1 students = [
2     (20251234, "김철수", "컴퓨터과학", 2, 3.8),
3     (20265678, "이영희", "생활과학부", 3, 4.2),
4     (20243456, "박민수", "사회복지학과", 1, 3.5)
5 ]
6 print("이름과 성적 출력:")
7 for  _ , name,  _ ,  _ , grade in students:
8     print(f"{name}의 성적: {grade}")
```

예외처리의 이해

- 코드 실행 중 예기치 않은 오류가 발생할 경우에 대비하는 기능 필요
  - 오류를 별도 처리하지 않는 경우 프로그램이 즉시 종료
  - 사용자 경험 저하, 데이터 손실, 리소스 누수, 디버깅 어려움 등의 추가적인 문제 발생
- 예외 및 예외 처리
  - 예기치 않은 오류를 예외(exception)로 정의
  - 예외 처리(exception handling)는 프로그램이 비정상적 종료를 방지
  - try-except 문과 finally 블록을 사용

try-except 문의 사용

- 오류가 발생할 가능성이 있는 코드 블록을 감싸서, 오류가 발생했을 때 프로그램이 강제 종료되지 않도록 처리

```
1 try:
2     number = int(input("숫자를 입력하세요: "))
3     result = 10 / number
4 except ZeroDivisionError:
5     print("0으로 나눌 수 없습니다.")
6 except ValueError:
7     print("유효한 숫자를 입력해야 합니다.")
```

- 여러 종류의 예외가 발생할 가능성이 있는 경우, except 블록을 여러 개 지정하여 각 예외를 개별적으로 처리

finally 블록의 사용

- 예외 발생 여부와 관계없이 반드시 실행되는 코드 블록

```
1  try:
2      file = open("data.csv", "r")
3      content = file.read()
4      print(content)
5  except FileNotFoundError:
6      print("파일을 찾을 수 없습니다.")
7  finally:
8      print("파일 작업 종료.")
9      file.close()
```

02

함수형 프로그래밍



함수형 프로그래밍의 이해

- 파이썬은 절차적 프로그래밍, 객체지향 프로그래밍 외 함수형 프로그래밍 방식을 지원
- 데이터 변경을 최소화하고, 순수 함수(pure function)를 활용하여 부작용(side effect)을 줄이는 것을 목표
  - 순수함수: 같은 입력에 대해 항상 같은 출력을 반환하는 함수
  - 부작용: 함수가 단순히 값을 계산하는 것 외에 외부에 영향을 미치는 모든 행위
- 코드의 가독성을 높이고, 디버깅을 용이하게 하며, 병렬 처리를 효율적으로 수행

람다 함수

- 이름이 없는 익명 함수(anonymous function)
  - 한 줄의 표현식으로 작성할 수 있는 간결한 함수
  - 즉시 사용할 수 있는 간결한 연산을 수행할 때 적합

문법형식

```
lambda 매개변수: 표현식
```

- 람다 함수의 문법적 특징
  - 단 하나의 표현식만 허용
  - 표현식의 결과는 자동으로 함수의 반환값이 되므로 return 키워드 사용하지 않음

람다 함수의 사용

일반 함수와 람다 함수

```
1 def add(x):
2     return x + 2
3 print(add(3))
```



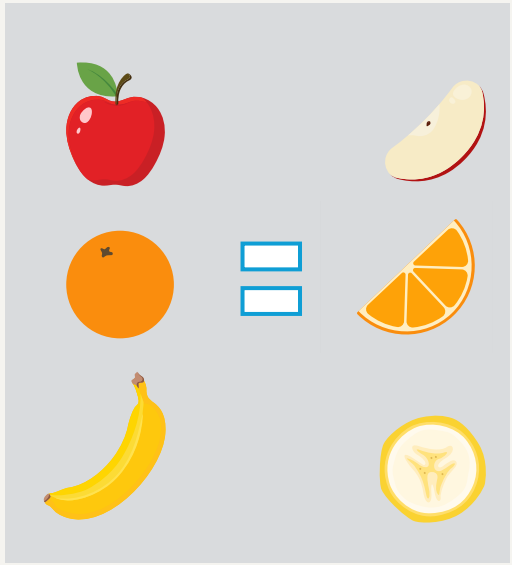
```
1 add_lambda = lambda x: x + 2
2 print(add_lambda(3))
```

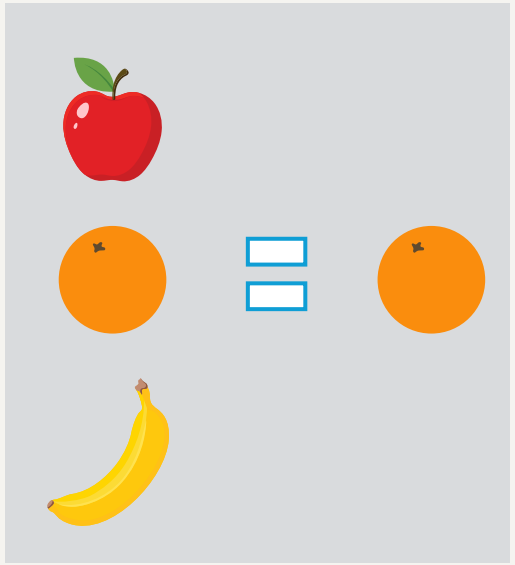
람다 함수의 활용

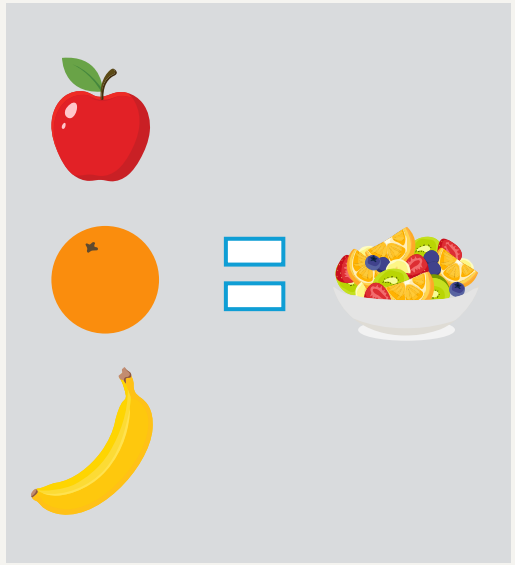
```
1 employees = { "이지혜": 3000000,
2               "구민준": 5000000,
3               "방서연": 4100000 }
4 adjust_salary = lambda salary: salary * 1.1
5 updated_salaries = {name: adjust_salary(salary) for
                      name, salary in employees.items()}
```

map, filter, reduce 함수

- 대표적인 함수형 프로그래밍 기법을 사용하는 함수
- 반복문을 사용하지 않고 리스트나 다른 반복 가능한 객체의 요소를 손쉽게 조작할 수 있도록 지원
  - 데이터 변환, 필터링, 누적 연산을 수행







map 함수

- 반복 가능한 객체의 모든 요소에 특정 함수를 적용하여 새로운 값을 생성하는 함수
  - 리스트나 튜플과 같은 자료구조의 요소를 변경할 때 유용

```
1 temperature_celsius = [25.6, 27.8, 30.5, 22.3, 28.9,
                         31.2, 24.7]
2 celsius_to_fahrenheit = lambda c: (c * 9/5) + 32
3 temperature_fahrenheit =
   list(map(celsius_to_fahrenheit, temperature_celsius))
4
5 print("섭씨 온도 데이터:", temperature_celsius)
6 print("화씨 온도로 변환:", temperature_fahrenheit)
```

filter 함수

- 반복 가능한 객체의 요소 중 특정 조건을 만족하는 요소만 남기는 함수
  - True 또는 False를 반환하는 함수(조건 함수)를 인자로 받고 조건 함수가 True를 반환하는 요소만 컬렉션에 포함

```
osda_students = [
    {"name": "김지원", "score": 85},
    {"name": "이민준", "score": 65},
    {"name": "박서연", "score": 90},
    {"name": "정현우", "score": 55},
    {"name": "최예은", "score": 78} ]
passed_students = list(filter(lambda student:
                student["score"] >= 70, osda_students))
print("70점 이상 합격한 학생 목록:")
for student in passed_students:
    print(f"이름: {student['name']}, 점수:
            {student['score']}")
```

reduce 함수

- 반복 가능한 객체의 모든 요소를 누적하여 하나의 값으로 축소하는 함수
  - 반복문을 사용하지 않고도 데이터를 하나의 값으로 집계
  - functools 모듈에서 제공

```
1 from functools import reduce
2 numbers = [1, 2, 3, 4, 5]
3 total_sum = reduce(lambda x, y: x + y, numbers)
4 print(total_sum)
```

① 첫 번째와 두 번째 요소를 함수에 적용하여 중간 결과를 생성
② 그 중간 결과와 객체의 다음 요소를 다시 함수에 적용
③ 이 과정을 반복하여 최종적으로 하나의 값을 반환

정리하기

- 언패킹은 시퀀스 자료형 요소를 개별 변수에 효율적으로 할당한다.
- 확장 언패킹(*)은 나머지 요소들을 리스트로 모을 수 있다.
- 언더스코어(_)는 값을 무시하거나 불필요한 반복 변수 대체에 활용한다.
- 예외 처리는 프로그램 비정상 종료 방지 및 오류 상황에 대응한다.
- 함수형 프로그래밍은 반복문 없이 데이터 변환, 필터링, 축소 수행으로 코드를 간결화하고 병렬 처리에 적합한 구조를 제공하여 데이터 처리 성능을 향상시킨다.
- 람다 함수는 간결한 익명 함수로 즉시 정의 및 사용이 가능하다.
- map은 시퀀스의 모든 요소에 함수를 적용하여 새로운 시퀀스를 생성한다.
- filter는 조건을 만족하는 요소만 선택한다.
- reduce()는 요소들을 순차적으로 처리하여 하나의 결과값으로 축약한다.

04강 데이터의 수집

다음시간안내

